C04 -AT1 -SIMATS COMPILER

Q1 **Hybrid Design in Smart Agriculture** 10 marks

A smart agriculture system uses inheritance for different farm types and composition for sensors. Analyze both approaches and implement a hybrid sustainable-farming system.

Your Code

```cpp
#include <iostream>
using namespace std;
class Farm {
protected:
    string name;
public:
    Farm(string n) {
        this->name = n;
    }
    virtual void show() = 0;
    virtual ~Farm() {}
};
class Sensor {
public:
    int moisture;
    Sensor(int m) {
        moisture = m;
    }
    void read() {
        cout << "Soil Moisture: " << moisture << "%\n";
    }
};
class SmartFarm : public Farm {
    Sensor sensor;
public:
    SmartFarm(string n, int m)
```

Input

```
Enter farm name: GreenFarm
Enter soil moisture: 30
```

Output

Visible Test Cases

Test Case 1

Q2 Method Chaining with this Pointer

10 marks

A carbon-tracking application supports method chaining for updating energy and emission values. Analyze how the this pointer enables chaining.

Your Code

```cpp
#include <iostream>
using namespace std;
class CarbonTracker {
    float energy, emission;
public:
    CarbonTracker& setEnergy(float e) {
        energy = e;
        return *this;
    }
    CarbonTracker& setEmission(float em) {
        emission = em;
        return *this;
    }
    void display() {
        cout << "Energy: " << energy << " kWh" << endl;
        cout << "Emission: " << emission << " kg CO2" << endl;
    }
};
int main() {
    float e, em;
    cin >> e >> em;
    CarbonTracker c;
    c.setEnergy(e).setEmission(em).display();
    return 0;
}
```

Input

```
120
45
```

Output

Visible Test Cases

Test Case 1

Q3 Virtual Base Class for Battery Data

10 marks

An EV battery-management system inherits common battery information through multiple classes, causing duplicate data. Analyze the issue and resolve it using a virtual base class.

Your Code

```cpp
#include <iostream>
using namespace std;
class Battery {
protected:
    int capacity;
public:
    void setCapacity(int c) {
        capacity = c;
    }
    void display() {
        cout << "Battery Capacity: " << capacity << " kWh";
    }
};
class BatteryInfo : virtual public Battery {
};
class BatteryStatus : virtual public Battery {
};
class EVBattery : public BatteryInfo, public BatteryStatus {
};
int main() {
    EVBattery b;
    int c;
    cin >> c;
    b.setCapacity(c);
    b.display();
    return 0;
}
```

Input

```
75
```

Output

Visible Test Cases

Test Case 1

Q4 **Runtime Polymorphism in Sustainability Dashboards** 10 marks

A sustainability dashboard processes SolarPanel, WindTurbine and Battery objects using a common base-class pointer. Analyze how runtime polymorphism supports different energy calculations.

Your Code

```cpp
#include <iostream>
using namespace std;
class Energy {
public:
    virtual void calculate() {
        cout << "Energy calculation" << endl;
    }
};
class SolarPanel : public Energy {
public:
    void calculate() override {
        int power, hours;
        cin >> power >> hours;
        cout << "Solar Energy: " << power * hours << " kWh" << endl;
    }
};
class WindTurbine : public Energy {
public:
    void calculate() override {
        int power, hours;
        cin >> power >> hours;
        cout << "Wind Energy: " << power * hours << " kWh" << endl;
    }
};
class Battery : public Energy {
public:
    void calculate() override {
```

Input

```
100 5
200 3
80 90
```

Output

Visible Test Cases

Test Case 1

Q5 **Dynamic Memory in E-Waste Management** 10 marks

An e-waste management system dynamically allocates recyclable products and releases them after processing. Analyze how new and delete work with constructors and destructors.

Your Code

```cpp
#include <iostream>
using namespace std;
class EWaste {
    string product;
public:
    EWaste(string p) {
        product = p;
        cout << "Constructor: " << product << endl;
    }
    ~EWaste() {
        cout << "Destructor: " << product << endl;
    }
    void display() {
        cout << "Recyclable Product: " << product << endl;
    }
};
int main() {
    string name;
    cin >> name;
    EWaste *e = new EWaste(name);
    e->display();
    delete e;
    return 0;
}
```

Input

```
Laptop
```

Output

Visible Test Cases

Test Case 1